AWS CLOUD SECURITY

AWS IAM Study Notes

Identity and Access Management — Architecture, Policies, Best Practices & Common Pitfalls

DATE	SCOPE	SECTIONS
March 31, 2026	Fundamentals to Production	9 Chapters · 7 Pitfalls

TABLE OF CONTENTS

1. Introduction
2. IAM Fundamentals
3. IAM Identities
4. IAM Policies
5. MFA & Cross-Account Access
6. IAM Access Analyzer
7. Limits & Quotas
8. Tips & Best Practices
9. Pitfalls & Remedies

1 Introduction

Identity and Access Management (IAM) is the cornerstone of AWS security architecture. Every interaction with AWS — whether launching an EC2 instance, reading from an S3 bucket, or updating a database — requires two things:

Authentication
Proving who you are to AWS

Authorization
Verifying what you are allowed to do

IAM provides the framework for both, enabling you to control access to AWS resources with precision and flexibility. Mastering IAM is fundamental to building secure, compliant, and scalable cloud architectures.

2 IAM Fundamentals

GLOBAL SERVICE

IAM is a global service that operates across all AWS Regions. Resources created in IAM — users, groups, roles, and policies — are automatically available worldwide.

Key IAM Principles

Least Privilege 1

Grant only the permissions required to perform a specific task.

Defense in Depth 2

Layer multiple security controls: MFA, network policies, encryption.

Separation of Duties 3

Distribute permissions across multiple identities to prevent abuse.

Regular Auditing 4

Continuously review and refine access permissions over time.

3 IAM Identities

IAM Users

Represents a person or application. Has unique credentials (password, access keys) with specific permissions.

IAM Groups

Collections of users sharing common permissions. Attach policies to groups for easier management.

IAM Roles

Identities assumed by trusted entities. No permanent credentials — provide temporary security credentials when assumed.

Types of Roles

1. **Service Roles**

   Allow AWS services to perform actions on your behalf.

2. **Cross-Account Roles**

   Enable access between AWS accounts — common in multi-account organizations.

3. **Federated User Roles**

   Allow users authenticated by external identity providers to access AWS.

4. **Service-Linked Roles**

   Predefined roles created automatically by AWS services.

Trust Policies

Every role has a **trust policy** (assume role policy) that defines who can assume the role:

```JSON
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Service": "ec2.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

4 IAM Policies

Policies are JSON documents that define permissions. They answer: *"What actions can be performed on which resources under what conditions?"*

Policy Structure Example

```JSON
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowS3ListAllBuckets",
            "Effect": "Allow",
            "Action": "s3:ListAllMyBuckets",
            "Resource": "*",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": "203.0.113.0/24"
                }
            }
        }
    ]
}
```

Policy Elements

`Version`

Policy language version — always use `"2012-10-17"`

`Statement`

Array of individual permission statements

`Sid` `optional`

Statement identifier for documentation purposes

`Effect`

Either `"Allow"` or `"Deny"`

`Principal`

Who the statement applies to (resource-based policies only)

`Action`

List of actions, e.g. `s3:GetObject`, `ec2:RunInstances`

`Resource`

ARNs of resources the statement applies to

`Condition` `optional`

Circumstances under which the statement applies

Service Control Policies (SCPs)

Part of AWS Organizations, SCPs set **maximum permissions** for accounts in an organization. They don't grant permissions — they only limit them.

```JSON
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Deny",
            "Action": "ec2:RunInstances",
            "Resource": "*",
            "Condition": {
                "StringNotEquals": {
                    "ec2:Region": ["us-east-1", "us-west-2"]
                }
            }
        }
    ]
}
```

WHAT THIS SCP DOES

This policy prevents launching EC2 instances outside `us-east-1` and `us-west-2`.

IAM Best Practices for Policy Design

- **Policy Variables** — enable dynamic policies that adapt to current context
- **Condition Keys** — add context-based access control for granular security
- **Identity Federation** — enables users to access AWS using external identity provider credentials, eliminating the need to create individual IAM users

5 MFA & Cross-Account Access

Multi-Factor Authentication (MFA)

MFA adds an extra layer of security by requiring two forms of authentication:

1

Something You Know

Your AWS password

2

Something You Have

A physical or virtual MFA device

ROOT ACCOUNT RULE

Always enable MFA on the root account — this is non-negotiable for security. Up to 8 MFA devices are supported per user for redundancy.

Cross-Account Access

Organizations frequently need to grant access across AWS accounts. Two primary patterns:

PATTERN 1 — ROLE ASSUMPTION (RECOMMENDED)

Account A `(123456789012)` assumes a role in Account B `(987654321098)` to gain temporary credentials.

PATTERN 2 — RESOURCE-BASED POLICIES

For services that support resource-based policies: S3, SQS, SNS, Lambda. Direct access without assuming a role.

APPROACH	CREDENTIALS	SERVICE SCOPE	RECOMMENDATION
Role Assumption	Temporary	Any service	Preferred
Resource-Based	Direct access	S3, SQS, SNS, Lambda	Limited use

6 IAM Access Analyzer

AWS IAM Access Analyzer helps identify resources shared with external entities and validates policies against best practices.

Key Features

1. **External Access Analysis** — Identifies resources accessible from outside your AWS account or organization
2. **Policy Validation** — Checks policies against best practices and identifies errors before deployment
3. **Policy Generation** — Analyzes CloudTrail logs to generate least-privilege policies based on actual usage

Common Findings

- Public S3 buckets
- IAM roles assumable by external accounts
- KMS keys with external key policies
- Lambda functions with cross-account permissions

IAM Policies for AWS Services

SERVICE ROLES

- Lambda needs a role to write logs to CloudWatch
- EC2 instances need a role to access S3
- ECS tasks need a role to pull images from ECR

SERVICE-LINKED ROLES (AUTO-CREATED)

- Auto Scaling: `AWSServiceRoleForAutoScaling`
- ElastiCache: `AWSServiceRoleForElastiCache`
- Cannot be modified — only deleted when the service no longer needs them

RESOURCE-BASED POLICIES

Supported by: S3 bucket policies, SNS topic policies, SQS queue policies, Lambda function policies, ECR repository policies.

7 IAM Limits & Quotas

Understanding IAM limits helps plan your architecture and avoid unexpected deployment failures.

RESOURCE	DEFAULT LIMIT	NOTES
Users per account	5,000	Use federation for larger organizations
Groups per account	300	Users can be in up to 10 groups
Roles per account	1,000	Soft limit — can be increased via support
Policies per user/group/role	10 managed	Plus inline policies (separate limits)
Managed policy size	6,144 chars	Compress or split policies if hitting limit
Inline policy size	2,048 chars	User inline; role/group inline = 10,240 chars
MFA devices per user	8	Multiple devices for redundancy

8 Tips & Best Practices

Security Best Practices

TIP 1

Never Use Root for Daily Operations

Reserve the root account for initial setup and billing only.

TIP 2

MFA Delete on S3 Buckets

Implement MFA Delete to protect against accidental object removal.

TIP 3

Use IAM Roles Everywhere

Roles provide temporary credentials, eliminating risk from exposed long-term access keys. EC2 instance profiles, Lambda execution roles, ECS task roles, CI/CD OIDC federation.

TIP 4

Rotate Credentials Regularly

Schedule automated rotation for all long-term access keys.

TIP 5

AWS Secrets Manager

Store and auto-rotate database passwords, API keys, and other secrets.

Policy Design Tips

TIP 6

Start with AWS Managed Policies

`ReadOnlyAccess`, `PowerUserAccess`, `ViewOnlyAccess` — then create custom policies to tighten or extend permissions.

TIP 7

Use Policy Conditions

Always add conditions where applicable for defense-in-depth access control.

TIP 8

Use NotAction Carefully

NotAction can grant broader-than-intended access — review scope before deploying.

TIP 9

Leverage Policy Validators

Use IAM Access Analyzer policy validation to catch errors before deployment.

TIP 10

Document Your Policies

Use Sid identifiers and inline comments to maintain clarity.

Operational Tips

TIP 11

AWS Organizations for Multi-Account

Centralize governance with Organizations and SCPs.

TIP 12

Implement Policy Versioning

Keep previous policy versions to enable quick rollback.

TIP 13

CloudWatch Logs Insights for Auditing

Query IAM-related CloudTrail events at scale.

TIP 14

Budget Alerts for IAM Operations

Monitor unexpected costs from IAM-related service usage.

TIP 15

Just-in-Time (JIT) Access

Grant elevated permissions only when needed, and revoke automatically.

Monitoring & Auditing Tips

TIP 16

Enable IAM Access Advisor

Track service-level permissions usage to remove unused access.

TIP 17

Real-Time Security Alerts

Create EventBridge rules for IAM changes, targeting SNS for alerts.

TIP 18

Separation of Duties

User Management → Security Team · Policy Creation → DevOps · Permission Assignment → Team Leads · Audit → Compliance Team

TIP 19

AWS Config for IAM Compliance

Use Config rules to continuously evaluate IAM configurations against standards.

TIP 20

IAM Hygiene Dashboard

Create a centralized view for dormant credentials, unused roles, and policy drift.

Enterprise IAM Architecture

MULTI-ACCOUNT ORGANIZATION WITH IAM IDENTITY CENTER

```tree
Management Account (Organizations)
├── IAM Identity Center
│   ├── Identity Source (Azure AD / Okta)
│   ├── Permission Sets
│   └── Account Assignments
├── Security Account (Audit / Logging)
├── Shared Services Account (Networking)
├── Development Accounts (by team)
└── Production Accounts (by application)
```

BENEFITS

Centralized identity management · Consistent permissions across accounts · Automatic account provisioning · Single sign-on experience

SCP HIERARCHY EXAMPLE

```tree
Root (FullAWSAccess)
├── Production OU
│   ├── RequireEncryption SCP
│   ├── DenyRootUsage SCP
│   ├── RegionRestriction SCP
│   └── RequireMFA SCP
├── Development OU
│   ├── RequireEncryption SCP
│   ├── DenyRootUsage SCP
│   └── RegionRestriction SCP
└── Sandbox OU
    └── DenyProductionAccess SCP
```

9 Pitfalls & Remedies

1 Overly Permissive Policies with Wildcard Resources

PROBLEM

Policies using `"Resource": "*"` with broad actions, granting unintended access across all resources.

WHY IT HAPPENS

- Quick prototyping without refinement
- Lack of understanding of resource-specific ARNs
- Copy-pasting examples without modification
- Pressure to "just make it work"

IMPACT

- Privilege escalation opportunities
- Lateral movement for attackers
- Compliance violations (least privilege principle)
- Difficulty tracking who has access to what

PREVENTION

- ✅ Always start with deny-all, then explicitly allow what's needed
- ✅ Use IAM policy simulator to test before deploying
- ✅ Implement a policy review process
- ✅ Set up automated scanning for overly permissive policies
- ✅ Use AWS Config rules to detect violations

2 Hardcoded Credentials in Code or Configuration Files

PROBLEM

Access keys embedded directly in application code, configuration files, or committed to version control.

WHY IT HAPPENS

- Convenience during development
- Lack of awareness of better alternatives
- Legacy applications not refactored
- Insufficient security training

IMPACT

- Exposed credentials if repository is public
- Credentials leaked in CI/CD logs
- Difficult to rotate without code changes
- Compliance failures and potential for massive breaches

PREVENTION

- ✅ Never commit credentials to version control
- ✅ Use IAM roles wherever possible
- ✅ Store secrets in AWS Secrets Manager or Parameter Store
- ✅ Implement automated scanning in CI/CD pipeline
- ✅ Rotate credentials regularly using automated tools

3 Root Account Usage for Daily Operations

PROBLEM

Using the root account for routine tasks instead of creating IAM users or using federated access.

WHY IT HAPPENS

- Convenience — it's the first account created
- Lack of understanding about IAM best practices
- Small teams without proper governance
- "It's just a test account" mentality that persists into production

IMPACT

- No audit trail of who performed actions
- Cannot restrict root account permissions
- Single point of compromise for entire account
- Compliance violations and risk of accidental destructive actions

PREVENTION

- ✅ Lock root credentials in a secure vault (password manager)
- ✅ Enable MFA on root account
- ✅ Set up CloudWatch alarms for root usage
- ✅ Use AWS Organizations SCP to restrict root usage
- ✅ Conduct regular reviews of root account activity

4 Insufficient Cross-Account Access Controls

PROBLEM

Cross-account roles configured without proper conditions, making accounts vulnerable to confused deputy attacks.

WHY IT HAPPENS

- Copying trust policies without understanding conditions
- Not using external IDs
- Overly permissive IP restrictions
- Lack of understanding of cross-account security models

IMPACT

- Unauthorized cross-account access
- Confused deputy vulnerability
- Lateral movement between accounts
- Compliance violations and difficult access tracking

PREVENTION

- ✅ Always use ExternalId for third-party access
- ✅ Specify exact principal ARNs (not `:root`)
- ✅ Add IP restrictions when possible
- ✅ Implement time-based access with conditions
- ✅ Use IAM Access Analyzer to detect external access
- ✅ Monitor AssumeRole calls in CloudTrail

5 Not Using SCPs in Organizations

PROBLEM

Managing permissions solely through IAM policies in individual accounts without leveraging AWS Organizations SCPs for centralized control.

IMPACT

- Inconsistent security postures across accounts
- Inability to enforce organization-wide policies
- Risk of rogue accounts with excessive permissions
- Compliance challenges at scale

PREVENTION

- ✅ Enable AWS Organizations for all multi-account setups
- ✅ Start with permissive SCPs and gradually tighten
- ✅ Test SCPs in sandbox accounts first
- ✅ Document the purpose of each SCP
- ✅ Review and update SCPs quarterly
- ✅ Use SCPs as guardrails, not primary access control

6 Neglecting IAM Policy Size Limits

PROBLEM

Creating overly complex policies that exceed AWS size limits, causing deployment failures or forcing workarounds that compromise security.

POLICY SIZE LIMITS

- Managed policy: **6,144 characters**
- Inline policy: **2,048 characters** (user) / **10,240** (role/group)
- Resource-based policy: varies by service

PREVENTION

- ✅ Design policies for scalability from the start
- ✅ Use wildcards and variables effectively
- ✅ Split large policies into logical modules
- ✅ Leverage ABAC for scalable access control
- ✅ Monitor policy sizes in CI/CD pipeline

7 Improper Handling of Temporary Security Credentials

PROBLEM

Treating temporary credentials from STS like permanent access keys — leading to expiration issues, hardcoded tokens, or failed renewals.

IMPACT

- Application outages when credentials expire
- Race conditions during credential refresh
- Security risks if temporary credentials are exposed
- Difficult debugging of intermittent failures

PREVENTION

- ✅ Use IAM roles wherever possible (EC2, Lambda, ECS)
- ✅ Implement automatic credential refresh
- ✅ Handle expiration errors gracefully
- ✅ Never store temporary credentials permanently
- ✅ Monitor CloudTrail for ExpiredToken errors
- ✅ Use AWS SDK credential providers
- ✅ Set appropriate session durations

Summary & Key Takeaways

IAM is the foundation of AWS security, controlling who can access which resources under what conditions. Mastering IAM requires understanding its components, implementing least privilege access, and following security best practices throughout the credential lifecycle.

IAM is Global

Resources created in IAM are available across all AWS Regions, simplifying management but requiring careful planning.

Roles Over Users

Prefer IAM roles with temporary credentials over long-term IAM user access keys to reduce security risk.

Least Privilege is Non-Negotiable

Grant only the minimum permissions required for each task, using fine-grained policies with specific resources and conditions.

Defense in Depth

Layer multiple security controls — MFA, IP restrictions, permission boundaries, and SCPs for comprehensive protection.

Cross-Account Requires Care

Use external IDs, specific principals, and conditions to prevent confused deputy vulnerabilities.

Federation Scales Better

For organizations with many users, implement SAML or OIDC federation instead of creating individual IAM users.

Audit Continuously

Use CloudTrail, IAM Access Analyzer, and automated compliance checks to detect and remediate security issues.

Know Your Limits

Understand IAM quotas (users, roles, policy sizes) to design scalable architectures without hitting hard limits.

FOUNDATION STATEMENT

Understanding IAM deeply enables you to build secure, scalable, and compliant AWS architectures. The patterns and practices in these notes form the security foundation for all subsequent services and solutions.